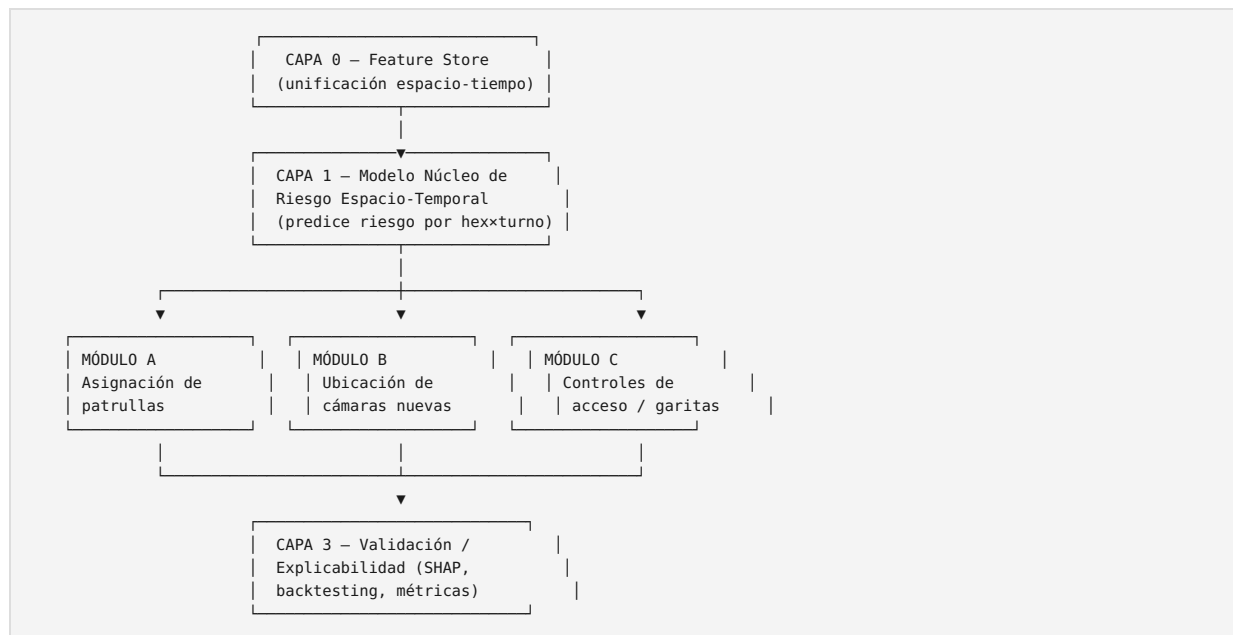


SIGE-BA — Arquitectura técnica del modelo predictivo de seguridad

0. Enfoque general

En vez de 4 modelos sueltos, conviene pensarlo como **1 modelo núcleo + 3 módulos de decisión** que consumen su output. Esto es más defendible frente al gobierno (“un solo motor de riesgo, auditable, que alimenta distintas decisiones operativas”) que 4 cajas negras separadas.



1. CAPA 0 — Unificación espacio-temporal (feature store)

1.1 Unidad espacial base

Se usa **grilla hexagonal H3** en vez de radio censal o barrio como unidad de modelado (aunque barrio/comuna se mantienen como capas de *reporte*, no de *modelado*).

- **Resolución H3-8** (~0,7 km² por hexágono, ~300 hexágonos cubriendo CABA) para el modelo núcleo — da suficiente densidad de eventos por celda para que el modelo no sea puro ruido.
- **Resolución H3-9** (~0,1 km², ~2.000 hexágonos) solo para visualización fina en el dashboard, no para entrenar.

Cada hexágono H3-8 se pre-calcula una sola vez con sus claves de agregación: `hex_id` → `radio_censal_id` → `barrio_id` → `comuna_id` (por centroide, point-in-polygon).

1.2 Unidad temporal base

`fecha` (día) × `turno` (Mañana 06-14 / Tarde 14-22 / Noche 22-02 / Madrugada 02-06).

Grano final de la tabla maestra: (**hex_id, fecha, turno**).

1.3 Pipeline de preprocesamiento (una sola vez, no en cada corrida)

1. Cargar `barrios.parquet`, `radios_censales.parquet`, `comunas` → generar índice espacial (geopandas sindex)
2. Generar grilla H3-8 sobre el polígono de CABA → tabla `hex_maestra` (`hex_id`, `geometry`, `centroid`, `barrio_id`, `radio_censal_id`, `comuna_id`)
3. Para cada dataset puntual (delitos, siniestros, camaras, alumbrado, etc.):
→ asignar `hex_id` vía `h3.geo_to_h3(lat, lon, resolution=8)`
4. Para calles (grafo): asignar `hex_id` a cada tramo por su punto medio; conservar jerarquía vial como atributo del hex
5. Para accesos_autopista: mapear a `hex_id` + tramo de calle troncal más cercano
6. Guardar todo como parquet intermedio en `data/features/` para no recalcularlo

2. Tabla de cruces (fuente → destino → tipo de join)

#	Fuente	Destino	Tipo de cruce	Feature resultante
1	delitos	hex_maestra	point-in-hex	conteo de delitos por hex×fecha×turno×tipo
2	delitos	radios_censales	point-in-polygon	NBI, hacinamiento, población del radio
3	siniestros_hechos	hex_maestra	point-in-hex	conteo de siniestros por hex×fecha×turno
4	siniestros_hechos	calles	nearest	jerarquía vial del lugar del siniestro
5	camaras	hex_maestra	point-in-hex	conteo cámaras actuales por hex
6	alumbrado	hex_maestra	point-in-hex + densidad	luminarias por hex, luminarias/km de calle
7	espacios_verdes (polígono)	hex_maestra	overlay % área	% superficie verde por hex
8	comisarias (polígono patrullaje)	hex_maestra	overlay	qué comisaría cubre cada hex hoy
9	comisarias_policia (punto)	delitos , hex_maestra	distancia	distancia a comisaría más cercana
10	estadios	hex_maestra	buffer 500m	flag “hex cerca de estadio”
11	eventos_masivos (2019, con lat/lon)	hex_maestra	point-in-hex + fecha	flag evento ese día en ese hex
12	eventos_masivos (2024-26, solo barrio)	barrio_id	join por barrio + fecha	flag evento ese día en ese barrio (resolución más gruesa)
13	clima	fecha (global, no espacial)	join por fecha	lluvia sí/no, temperatura, ese día para toda la ciudad
14	escuelas , hospitales , universidades , cajeros	hex_maestra	buffer 150m	conteo de POIs sensibles cerca del hex
15	ecobici_estaciones + viajes_agregado	hex_maestra	point-in-hex + agregado	flujo de personas estimado por hex×franja
16	molinetes_estaciones + agregado	hex_maestra	point-in-hex + agregado	flujo de pasajeros de subte por hex×franja
17	población (comuna/barrio)	hex_maestra	join por barrio, prorrateado por área	población estimada por hex → denominador para tasas per cápita
18	accesos_autopista	calles (jerarquía troncal)	nearest + filtro jerarquía	nodo de entrada a corredores viales principales

3. CAPA 1 — Modelo núcleo de riesgo espacio-temporal

3.1 Target

Dos alternativas, se recomienda la primera para arrancar:

- **Conteo esperado de incidentes** por (hex, fecha, turno, tipo_delito agrupado) → regresión con distribución Poisson/Binomial Negativa (maneja bien conteos con muchos ceros, que van a ser la mayoría de las celdas)
- Alternativa más simple para el primer prototipo: **tasas de riesgo relativa** (percentil de riesgo 0-100 por hex×turno), más fácil de explicar en la demo aunque menos granular

3.2 Algoritmo recomendado

Gradient Boosting (XGBoost o LightGBM) con objetivo Poisson, no red neuronal ni deep learning espaciotemporal:

- Corre 100% local, gratis, entrena en minutos con este volumen de datos
- Da `feature_importance` y es compatible con SHAP → clave para el pitch de “no es caja negra”
- Maneja bien variables heterogéneas (categóricas + continuas + geoespaciales agregadas) sin necesitar normalización compleja

3.3 Features de entrada (resumen, grano hex×fecha×turno)

--	--

Grupo	Features
Histórico propio	conteo de delitos en el mismo hex, lags de 7/30/365 días; conteo del mismo hex mismo turno hace 1 semana/mes/año
Vecindad espacial	conteo de delitos en hexágonos vecinos (anillo H3 k=1, k=2) — “riesgo de contagio espacial”
Socioeconómico	NBI, hacinamiento (de <code>radios_censales</code>), población del hex
Infraestructura	densidad de alumbrado, % espacio verde, distancia a comisaría, conteo de cámaras actuales, jerarquía vial del hex
Flujo/exposición	flujo ecobici, flujo molinetes (proxy de cuánta gente circula)
Contexto puntual	POIs sensibles cerca (escuelas, cajeros, hospitales)
Temporal/calendario	día de semana, franja horaria, mes, feriado (derivable de fecha)
Exógeno	lluvia/temperatura del día, flag evento masivo, flag partido en estadio cercano

3.4 Validación

- **Split temporal, no aleatorio:** entrenar hasta 2023, validar con 2024, testear con 2025 (nunca mezclar años para no filtrar información futura)
- **Validación espacial adicional:** dejar afuera un subconjunto de hexágonos completos para chequear que el modelo generaliza a zonas no vistas
- **Métricas:**
 - Para el conteo: MAE/RMSE, y comparación contra baseline naive (promedio histórico del mismo hex-turno)
 - Para el ranking de riesgo: **Recall@K** — de los K hexágonos que el modelo marca como más riesgosos, cuántos incidentes reales del período de test caen ahí (esta es la métrica que más importa para la demo: “el modelo concentra el X% de los delitos reales en el Y% del área que marca como prioritaria”)

3.5 Output

Tabla `riesgo_predicho` (`hex_id`, `fecha`, `turno`, `tipo_delito`, `score_riesgo`, `conteo_esperado`) — esto es lo único que consumen los 3 módulos de abajo. Ningún módulo vuelve a tocar los datos crudos.

4. CAPA 2 — Módulos de decisión (optimización, no predicción)

Estos ya no son modelos de ML — son problemas de **optimización con restricciones**, resueltos con `pulp` o `scipy.optimize` (gratis, corre local).

4.1 Módulo A — Asignación de patrullas

- **Problema:** *Maximal Covering Location Problem* — dado un número K de móviles disponibles (parámetro, el slider del dashboard) y un radio de cobertura efectiva R (ej. 800m en zona urbana densa), elegir las K ubicaciones que maximizan el riesgo total cubierto
- **Inputs:** `riesgo_predicho` (filtrado por turno elegido), población por hex, comisarías actuales como puntos candidatos base + hexágonos de alto riesgo como candidatos adicionales
- **Restricción realista:** no reasignar el 100% del mapa — anclar un piso mínimo de cobertura en cada comuna (para que no se pueda alegar “dejaron una comuna entera sin patrullaje”)
- **Output:** lista de ubicaciones + cuántos móviles por turno, y el % de riesgo total cubierto vs. la distribución actual (esta comparación es el argumento central de venta)

4.2 Módulo B — Ubicación de cámaras nuevas

- **Problema:** *Weighted Set Cover* — dado un presupuesto de N cámaras nuevas, elegir ubicaciones que maximicen riesgo cubierto, penalizando zonas ya cubiertas por `camaras.parquet` actual
- **Ponderadores adicionales:** baja densidad de alumbrado (prioridad), alto flujo peatonal (ecobici/molinetes), sin cámara existente en radio de 100m
- **Output:** ranking de ubicaciones candidatas ordenado por “ganancia marginal de cobertura”, no solo un mapa de calor

4.3 Módulo C — Controles de acceso / garitas

- **Problema distinto:** acá se trabaja sobre el **grafo vial** (`calles` + `accesos_autopista`), no sobre hexágonos sueltos
- **Método:** desde cada acceso, recorrer los tramos troncales/distribuidores (filtrando por jerarquía) y rankear por densidad combinada de `siniestros_hechos` + `riesgo_predicho` de los hexágonos que atraviesa ese corredor
- **Output:** top-N ubicaciones de control, cada una con su justificación (accidentalidad histórica + riesgo delictivo del

5. CAPA 3 — Validación y explicabilidad

- **SHAP values** sobre el modelo núcleo → para cada predicción de alto riesgo, poder mostrar “esto se explica en un 40% por historial reciente, 25% por baja iluminación, 20% por proximidad a evento masivo, etc.” — esto es lo que hace la diferencia entre un modelo que impresiona y uno que genera desconfianza
- **Backtesting narrado**: tomar un mes real de 2025 (fuera de entrenamiento), correr el modelo “como si fuera hoy”, y comparar contra lo que realmente pasó — este es el mejor material posible para la demo, mucho más convincente que un mapa de calor sin contraste
- **Dashboard de métricas del propio modelo** (no solo el mapa operativo) — un panel aparte con Recall@K, curva de calibración, y evolución mes a mes, para mostrar rigor técnico

6. Stack técnico (costo \$0)

Capa	Herramienta
Procesamiento geoespacial	geopandas , h3-py
Modelo núcleo	xgboost o lightgbm (objetivo Poisson)
Explicabilidad	shap
Optimización (módulos A/B/C)	pulp (programación lineal entera, gratis)
Almacenamiento intermedio	parquet (ya lo estás usando)
Orquestación local	scripts Python ejecutados vía Claude Code
Frontend	Next.js + React, desplegado en Vercel (plan Hobby, gratis)
Mapas	react-map-gl / deck.gl (free tier) o PMTiles estático
Repositorio	GitHub (gratis, privado si preferís)

7. Estructura de carpetas sugerida para Claude Code

data/	
processed/	# tus 32 parquet originales, ya está
features/	# tablas intermedias post-cruces (hex_maestra, riesgo_predicho, etc.)
notebooks/	# exploración inicial, un notebook por capa
src/	
etl/	# scripts de la Capa 0 (unificación espacial/temporal)
model_core/	# entrenamiento y predicción del modelo núcleo (Capa 1)
optimization/	# módulos A, B, C (Capa 2)
validation/	# backtesting, SHAP, métricas (Capa 3)
export/	# genera los JSON/GeoJSON livianos para el dashboard
dashboard/	# proyecto Next.js separado, consume /data/features/export/*.json

8. Roadmap de implementación (orden sugerido)

1. **Capa 0**: pipeline de unificación espacial (grilla H3 + claves) — es la base de todo lo demás, sin esto no se puede avanzar
2. **Capa 1 v1**: modelo núcleo con features históricas + socioeconómicas únicamente (sin exógenas todavía) — para tener un primer resultado rápido y validar que el approach funciona
3. **Capa 1 v2**: sumar variables exógenas (clima, eventos, estadios) y medir si mejora el Recall@K respecto a v1 — esto también es un lindo dato para el pitch (“agregar eventos masivos mejoró la predicción en X%”)
4. **Módulo A** (patrullas) — el más directo de construir sobre el output de Capa 1
5. **Módulos B y C** (cámaras, controles) en paralelo, mismo patrón
6. **Capa 3** (SHAP + backtesting) — se hace en paralelo a medida que cada pieza está lista, no al final
7. **Export + dashboard** — última etapa, una vez que los números están validados

¿Empezamos a bajar esto a scripts concretos (por ejemplo, el pipeline de la Capa 0) para que se lo pases directo a Claude Code, o preferís que primero revisemos si esta arquitectura te cierra en algún punto puntual?